

Il modello di rumore per il trimero ad anello

Stesso modello del dimero, riusato su un circuito a 3 qubit

Note di lavoro — Tesi triennale, Samuele

Relatore: Prof. Alessandro Chiesa

File di codice: `noise_model_trimero_anello.py` (nuovo, wrapper sottile su `noise_model_dimero.py` — Sez. 2), `validate_noise_model_trimero_anello.py` (nuovo, verifica l'aggancio a 3 qubit). Derivazione completa delle formule in `teoria_modello_rumore.tex` (comune al dimero, non richiama nulla di specifico a $N = 2$). Nessun punto di lavoro specifico per il modello in sé: come per il dimero, è la base condivisa da tutti gli stadi successivi della pipeline.

Indice

1 Obiettivo	1
2 Riuso del modulo: perché è diretto, non un'estensione	1
3 Verifica	2
4 Perché 6 CNOT, e perché è indipendente da ε_{2q}	4
5 Cosa non entra in questo modello	4
6 Considerazioni e cosa serve al passo successivo	4
File prodotti	5

1 Obiettivo

Aprire la Parte 2 (rumore) sul trimero ad anello, dopo la chiusura della Parte 2 sul dimero (relazione inviata, riscontro positivo, `log_decisioni.md`, 6 settembre 2026). Il relatore ha confermato la topologia (anello, non catena, stessa data) e lo scope dei canali di rumore era già stato fissato per il dimero (`domande_relatore.md`, sez. 12): **due soli canali**, errore di gate (depolarizzante, 1 e 2 qubit) ed errore di lettura (readout, simmetrico come primo passo). T_1/T_2 esclusi, stessa decisione esplicita.

Questo documento non deriva un nuovo modello: verifica che quello già costruito e validato per il dimero si applichi correttamente a un sistema a 3 spin, senza modifiche.

2 Riuso del modulo: perché è diretto, non un'estensione

Il modulo `noise_model_dimero.py` costruisce il `NoiseModel` con due chiamate:

```
nm.add_all_qubit_quantum_error(err, [...])
nm.add_all_qubit_readout_error(ro_err)
```

Entrambe applicano l'errore a *tutti* i qubit del circuito su cui il `NoiseModel` viene agganciato in fase di simulazione, quale che sia il loro numero: non c'è alcun riferimento hard-coded a 2 qubit nel modulo. Il riuso per l'anello (3 qubit) è quindi diretto, non un'estensione da scrivere — verificato empiricamente in Sez. 3, non solo argomentato dalla lettura del codice.

Nota di dipendenza, dichiarata esplicitamente

`noise_model_trimero_anello.py` è un **wrapper sottile**: importa e ri-espone tal quale (`build_noise_model`, `eps_to_lambda`, `BASIS_GATES` e le costanti di calibrazione) da `noise_model_dimero.py`, senza ridefinire alcuna logica. Stessa situazione, dichiarata con lo stesso principio, già segnalata per `validate_ansatz_params_dimero.py` in Parte 1 del dimero: un file concettualmente di un'altra parte del progetto viene riusato da un contesto diverso, e la dipendenza è dichiarata nel docstring del modulo, non nascosta dietro un import silenzioso. Il motivo del wrapper, invece di un import diretto da `validate_noise_model_trimero_anello.py`: dare al trimero un punto d'ingresso con lo stesso nome usato in ogni altra parte del progetto (`noise_model_dimero.py`, e in futuro `noise_model_trimero_catena.py` se servirà), così che il pacchetto riproducibile del trimero elenchi un `noise_model_trimero_anello.py` accanto agli altri moduli senza dover spiegare ogni volta l'assenza — senza però duplicare la logica in due copie da tenere sincronizzate.

Le formule del canale depolarizzante e dell'errore di lettura, e la calibrazione di riferimento, sono identiche a quelle già derivate per il dimero (`teoria_modello_rumore.tex`, `risultati_modello_rumore_dimero.tex`) — non ripetute qui per esteso, solo richiamate dove servono per la verifica.

Canale	Gate di riferimento	Valore mediano
Errore di gate, 1 qubit	<code>sx</code>	$\varepsilon_{1q} = 2.9 \times 10^{-4}$
Errore di gate, 2 qubit	<code>CZ</code>	$\varepsilon_{2q} = 3.8 \times 10^{-3}$
Errore di lettura	—	$p_{\text{readout}} = 2.3 \times 10^{-2}$

Fonte: J. P. T. Stenger, G. Bazargan, N. T. Bronn, D. Gunlycke, “*Method for simulating open-system dynamics using mid-circuit measurements on a quantum computer*”, arXiv:2504.15187 (2025) — Appendice B, mediane pubblicate per `ibm_torino`.

3 Verifica

Punto di lavoro usato per la verifica: VQE+DM dell'anello, ansatz W -2q.6, $J = 1$, $J' = 0.4$, $b = b_c = 2.4$, $D = 0.15$ (punto confermato, mode “B”). Parametri VQE riottimizzati in questa sessione (multistart COBYLA + polish L-BFGS-B, stesso schema del dimero): $E_{\text{VQE}} = -5.53437001739340$, $E_{\text{esatto}} = -5.53437001739350$, fedeltà = 0.999999999999961 — coincide con la fidelity ≈ 1 già registrata in Parte 1 (Fase 5, ansatz W -2q.6).

Controllo 1 — limite di rumore nullo

$\varepsilon_{1q} = \varepsilon_{2q} = p_{\text{readout}} = 0$: l'energia calcolata su `AerSimulator(method="density_matrix")` deve riprodurre l'energia VQE di Parte 1.

Quantità	Valore
E_{VQE} (registrato, Parte 1)	-5.5343700174
E_{VQE} (density_matrix, rumore=0)	-5.5343700174
differenza	2.66×10^{-15}
CNOT nel circuito transpilato	6

OK: differenza a precisione di macchina, ben sotto la soglia 10^{-8} richiesta.

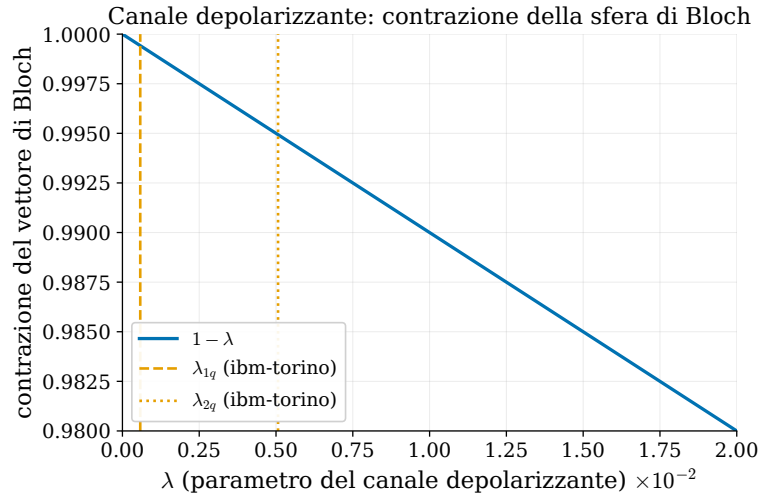


Figura 1: Contrazione del vettore di Bloch, $1 - \lambda$, in funzione di λ — stesso contenuto del documento sul dimero: la fisica del canale depolarizzante a 1 qubit non dipende dal numero di spin del sistema più ampio in cui il qubit è inserito. **Nota tecnica:** il PDF originale del Project, letto in questa sessione, risulta un archivio zip con anteprima JPEG (limite della piattaforma per i file binari montati in `/mnt/project` in questo ambiente — verificato su più PDF, non isolato a questo file — non un difetto del file sorgente). Rigenerata qui come vero PDF vettoriale (`generate_figure_depolarizzazione_bloch.py`, stesso stile grafico del progetto via `stile.py`) a partire dalla formula nota ($1 - \lambda$) e dalle etichette/range esatti recuperati dal testo estratto nel manifest dello stesso pacchetto zip, non inventati.

Controllo 2 — NoiseModel con valori di riferimento (ibm_torino)

Quantità	Valore
E_{VQE} (rumore di riferimento)	-5.37687489
Scostamento da E_{VQE} ideale	+0.157495
CNOT nel circuito transpilato	6

OK: energia rumorosa più alta del fondamentale, come atteso fisicamente per un canale che degrada verso lo stato massimamente misto.

Controllo 3 — scan rapido su ε_{2q}

A parità di $\varepsilon_{1q} = 2.9 \times 10^{-4}$, $p_{\text{readout}} = 2.3 \times 10^{-2}$:

ε_{2q}	E_{VQE} rumoroso	scostamento	CNOT
0	-5.49600915	+0.038361	6
1.0×10^{-3}	-5.46445616	+0.069914	6
3.8×10^{-3}	-5.37687489	+0.157495	6
8.0×10^{-3}	-5.24760282	+0.286767	6
1.5×10^{-2}	-5.03765731	+0.496713	6

OK: scostamento monotono crescente in ε_{2q} , come atteso. Il conteggio di CNOT resta 6 per ogni valore — stesso pattern già osservato sul dimero (il conteggio dopo transpilazione dipende dalla struttura del circuito, non dai valori numerici dei parametri del rumore né, qui, da quelli dell'ansatz).

4 Perché 6 CNOT, e perché è indipendente da ε_{2q}

Il circuito VQE+DM dell'anello (ansatz *W-2q.6*) contiene tre blocchi *W* (uno per legame: (1, 2), (2, 3), (3, 1)), ciascuno costruito come $\text{cx} \cdot R_y(\theta) \cdot \text{cx}$ — 2 CNOT per blocco, $3 \times 2 = 6$ in totale, più le rotazioni a 1 qubit finali che non aggiungono CNOT. La transpilazione (`optimization_level=3`) non trova ulteriori fusioni: a differenza del blocco di Trotter del dimero (dove passi consecutivi identici possono fondersi), qui i tre blocchi agiscono su coppie di qubit diverse e in sequenza fissa, quindi non c'è struttura ripetuta da collassare. Il conteggio non dipende da ε_{2q} per lo stesso motivo per cui non dipendeva dai valori di θ sul dimero: il `NoiseModel` agisce *dopo* la transpilazione, sul circuito già fissato — cambia la probabilità di errore per ciascun gate, non la struttura del circuito stesso.

5 Cosa non entra in questo modello

Identico al dimero, per coerenza esplicita: T_1/T_2 esclusi (decisione del relatore); nessuna correlazione fra qubit (ogni canale applicato indipendentemente qubit per qubit, o coppia per coppia per il `cx`); readout simmetrico soltanto in questa fase base; calibrazione a un solo punto nel tempo (mediane pubblicate).

6 Considerazioni e cosa serve al passo successivo

Il riuso diretto del modulo (Sez. 2) significa che lo Stadio 2 (VQE con DM sotto rumore) non richiede nuova infrastruttura di *rumore*: il `NoiseModel` è già pronto, `energia_vqe_su_simulatore(noise_model, params)` già scritta in `validate_noise_model_trimero_anello.py` ne fissa lo schema d'uso (transpilazione + simulatore a operatore densità). La funzione di costo effettiva dello Stadio 2 non è però un riuso letterale di quest'ultima: aggiunge il calcolo della fedeltà (assente qui, dove serve solo l'energia per il controllo 1), quindi è una funzione nuova ma strutturalmente analoga — stesso schema del dimero, dove `validate_noise_model_dimero.py` e `vqe_dm_rumoroso_dimero.py` sono due funzioni distinte per lo stesso motivo, non la stessa funzione riusata (verificato ora sul codice, non assunto per analogia).

A differenza del dimero, però, la strategia di transpilazione a blocchi per Trotter e correlatori (Stadi 3–4) va scritta da zero: verificato esplicitamente che né `circuito_correlazioni_trimero_anello.py` né `trotter_trimero_anello.py` contengono riferimenti a `BASIS_GATES` — non un'estensione di codice esistente, un modulo nuovo da scrivere seguendo lo stesso principio già

fissato sul dimero (transpilare il singolo passo/blocco una volta, comporre N volte).

Sintesi in una riga

Stesso modello di rumore del dimero (due canali, calibrazione `ibm_torino`), riusato senza modifiche su un circuito a 3 qubit — verificato con tre controlli su ansatz W -2q.6, non assunto dall'analogia con $N = 2$.

File prodotti

`noise_model_trimero_anello.py` (wrapper sottile su `noise_model_dimero.py`, Sez. 2), `validate_noise_model_trimero_anello.py`, questo documento.